INDEX

SABHARI SAINATH A.M, II/IT, II SEM,

NAME: _____ STD.: _____ SEC.: _____ ROLL NO.: _____ SUB.: _____

DAA CAB, (OBSERVATION NOTE)

S. No.	Date	Title	Page No.	Teacher's Sign / Remarks
		I - COUNTER METHOD		
01	6/8/26	Finding Complexity-1		
02	6/8/26	Finding Complexity-2		
03	6/8/26	Finding Complexity-3		
04	6/8/26	Finding Complexity-4		
05	6/8/26	Finding Complexity-5		
		II - Divide and Conquer method		
06	20/8/26	Find the number of zeroes in an array		
07	20/8/26	Majority element		
08	20/8/26	Finding Floor value		
09	20/8/26	Two elements sum to x		
10	20/8/26	Sorting (Insertion)		
	III	Greedy Algorithms		
11	27/8/26	Coin Problem		
12	27/8/26	Cookies Problem		
13	27/8/26	Burgers Problem		
14	27/8/26	Array sum max		
15	27/8/26	Product of array elements		

S. No.	Date	Title	Page No.	Teacher's Sign / Remarks
		(DP)		
16	10/10/26	1.DP Playing with numbers.		
17	10/10/26	2.DP Playing with chessboard		
18	10/10/26	3.DP Largest common subsequence		
19	10/10/26	4.DP Largest non decreasing sequence.		
		(Competetive Programming)		
20	10/10/26	1.CP Duplicate O(N) O(1)		
21	10/10/26	2.CP Finding duplicates		
22	10/10/26	3.CP Print Intersection of 2 sorted arrays		
23.	10/10/26	4.CP Print Intersection of 2 sorted arrays		
24.	10/10/26	5.CP Pair with difference		
25.	10/10/26	6.CP Pair with difference		

Completed

6/8/26

Finding Complexity Using Counter method - 1

Aim:

To find the counter variable and time complexity.

Algorithm:

```
void function (int n)
{
    int i=0;
    int s=1;
    while (s<=n)
    {
        i++;
        s+=i;
    }
}
```

Program:

```
#include<stdio.h>
int main()
{
    int n, count=0;
    scanf("%d",&n);
    int s=1; count++;
    int i=1; count++;
    while (s<=n, count++)
    {
        i++; count++;
        s+=i; count++;
    }
    printf("%d", count);
}
```

Result:

Time Complexity:

linear loop, hence O(n).

Sample Input:	Sample output:	Got
9	12	12
4	9	9

Result:

Thus the time complexity and counter variable is found for the given algorithm.

6/8/26

Exp:02

Finding Complexity using Counter method - 2

Aim:

To find the time complexity for a given algorithm using counter method.

Algorithm:

```
##
{
  if(n ==1)
  {printf("*");
  }
  else
  {
    for(int i=1; i<=n; i++)
      {
        for(int j=1; j<=n; j++)
          {
            printf("*");
            printf("+");
            break;
          }
      }
  }
}
```

Program:

```
#include<stdio.h>
int main()
{
  int n;
  scanf("%d", &n);
  int count =0;
  if(n==1)
  {
    count++;
  }
```

```
else
{
    count++;
    for(int i=1; count++, i<=n; i++, count++)
    {
        for(int j=1; count++, j<=n; j++, count++)
        {
            count++;
            count++;
            break;
        }
    }
}
printf("%d", counter);
}
```

Time Complexity:

two for loops (nested)

$\therefore = O(n^2)$

Sample input	Sample output	got
2	12	12
1000	5002	5002

Result:

Thus the time complexity and counter variable is found for the given algorithm.

6/8/26
Exp 03

Finding Complexity using Counter variable method - 3

Aim:

To find the counter variable and complexity for the given algorithm.

Algorithm:

```
for(int i=1; i<=num; ++i)
{
    if(num%i==0)
    {
        printf("%d", i);
    }
}
```

Program:

```
#include<stdio.h>
int main()
{
    int num;
    scanf("%d", &num);
    int count=0;
    int i=0;
    for(i=1; count++, i<=num; ++i, counter++)
    {
        if(num%i==0)
        {
            count++;
        }
    }
    printf("%d", counter);
}
```

Sample input	Sample output	got
12	31	31
25	54	54

Time Complexity:

$O(n)$ ∵ single for-loop.

Result:

6/8/26
Exp: 04

Finding complexity using counter method - 4

Aim:

To find the tire complexity and counter variable for the given algorithm.

Algorithm:

```
void func(int n)
{
  int c=0;
  for(int i=n/2; i<n; i++)
    for(int j=1; j<n; j=2*j)
      for(int k=1; k<n; k=k*2)
        c++
}
```

Program:

```
#include<stdio.h>
int main()
{
  int n;
  scanf("%d",&n);
  int c=0
  int count=0;
  count++;
  for(int i=n/2; count++, i<n; i++)
  {
    for(int j=1; count++, j<n; j=j*2)
    {
      for(int k=1; count++, k<n; k=k*2)
      {
        c++;
      } count++;
  }}
} printf("%d",count);
```

Sample input	Sample output	Got
4	30	30

Time Complexity: $O(n^3)$ (∵ 3 nested for-loops)

Result:

Thus counter variable and time complexity are

6/8/26
exp 05

Finding complexity for an algorithm using counter method - 5

Aim:

To find the tire complexity and counter variable for a given algorithm.

Algorithm:

```
void reverse(int n)
{
  int rev=0, remainder;
  while(n!=0)
  {
    remainder = n%10;
    rev = rev*10 + remainder;
    n/=10;
  }
} printf(rev);
```

Program:

```
#include<stdio.h>
int main()
{
  int n;
  scanf("%d",&n);
  int count=0;
  int rev=0,remainder; count++;
  while(count++, n!=0)
  {
    remainder = n%10; count++;
    rev = rev*10 + remainder; count++;
    n = n/10; count++;
  }
  count++;
} printf("%d",count);
```

Sample input	Sample output	Got
12	11	11

Time complexity: $O(\log n)$ (∵ single while loop)

Result:

Thus the time complexity and counter variable are found for the given algorithm

Exp06
20/08/26

Divide And Conquer

Find the number of zeroes in an Array

Aim:

To find the number of zeroes in an array using divide and conquer method.

Program:

```
#include<stdio.h>
int main()
{
  int n;
  scanf("%d", &n);
  int a[n];
  for(int i=0; i<n; i++)
  {
    scanf("%d", &a[i]);
  }
  int count = 0;
  for(int i=0; i<n; i++)
  {
    if(a[i]==0)
    {
      count++;
    }
  }
  printf("%d", count);
}
```

Input	Expected	Got
5 1 1 0 0	2	2

Res

Result:

Thus the number of zeroes in an array is found using divide and conquer method.

Exp 07
08/20/26

Divide and Conquer

Majority Element.

Aim:

To find the majority occurring element in an array using divide and conquer method.

Program:

```
#include<stdio.h>
int main()
{
  int n;
  scanf("%d",&n);
  int a[n];
  for(int i=0;i<n;i++)
  {
    scanf("%d",&a[i]);
  }
  int ct=0;
  for(int i=0;i<n;i++)
  {
    for(int j=0;j<n;j++)
    {
      if(a[i]==a[j])
      {
        ct++;
      }
    }
    if(ct>n/2)
    {
      printf("%d",a[i]);
      break;
    }
  }
}
```

Input	Expected	Got
3 3 2 3	3	3

Result:

Thus the majority occurring element in an array is found using divide and conquer method.

Finding Floor value
using Divide and Conquer method

To print the floor value for the element in the sorted array

Program:

```
#include <stdio.h>
int main()
{
  int n;
  scanf("%d",&n);
  int a[n];
  for(int i=0;i<n;i++)
   { scanf("%d",&a[i]);
   }
  int key;
  scanf("%d",&key);
  for(int i=n;i>=0;i--)
  {
    if(a[i]<=key)
    {
      printf("%d",a[i]);
      break;
    }
  }
}
```

Input	Expected	Got
n = 6 1 2 8 10 12 19 key = 5	2	2

Result:

Thus the floor value using Divide and Conquer method is found.

Exp 09:
20/08/24

Two Elements sum to X using Divide and Conquer

Aim:

To find the elements that sum up to the given target X.

Program:

```
#include <stdio.h>
int main()
{
  int n;
  scanf("%d", &n);
  int a[n];
  for(int i=0; i<n; i++)
  {
    scanf("%d", &a[i]);
  }
  int key;
  int c=0, d=0;
  scanf("%d", &key);
  for (int i=0; i<n; i++)
  {
    for (int j=0; j<n; j++)
    {
      if (a[i]+a[j]==key)
      {
        c = a[i];
        d = a[j];
        break;
      }
    }
  }
  int b[2];
  if (c > d)
  {
    b[0]=c;
    b[1]=d;
  }
  else
  {
    b[0]=d;
    b[1]=c;
  }
  if (c!=0 && d!=0)
  {
    printf("%d %d\n", b[0], b[1]);
  }
  else
  {
    printf("No");
  }
}
```

Input	Expected	Got
n= 4 2 4 8 10 key 14	4 10	4 10

Result:

Thus the two number that sum upto target x is found using divide and conquer.

Exp 10
20/8/26

Implementation of quick sort.

Aim:

To do sorting of arrays.

Program:

```
#include<stdio.h>
int main()
{
  int n;
  scanf("%d", &n);
  int a[n];
  for(int i=0; i<n; i++)
  {
    for(int j=0; j<n-1-i; j++)
    {
      if(a[j]>a[j+1])
      {
        int temp= a[j+1];
        a[j+1] = a[j];
        a[j] = temp;
      }
    }
  }
  for(int i=0; i<n; i++)
  {
    printf("%d ", a[i]);
  }
}
```

Input	Expected	Got
5 67 34 12 98 78	12 34 67 78 98	12 34 67 78 9

Result:

Thus the given array is sorted.

20/08/26

Implementation of quick sort

Aim:

To perform quick sort for sorting the array.

Program:

```
#include<stdio.h>

void qsort(int a[], int l, int h)
{
  if(l<h)
  {
    int i = l;
    int j = h;
    int pivot = a[l];
    while(i<j)
    {
      while(a[i]<=pivot && i<h)
      {
        i++;
      }
      while(a[j]>pivot)
      {
        j--;
      }
      if(i<j)
      {
        int temp = a[i];
        a[i] = a[j];
        a[j] = temp;
      }
    a[l] = a[j];
    a[j] = pivot;
    qsort(a, l, j-1);
    qsort(a, j+1, high);
    }
  }

int main()
{
  int n, i;
  int a[100];
  scanf("%d", &n);
  for(int j=0; j<n; j++)
  {
    scanf("%d", &a[j]);
  }
qsort(a, 0, n-1);
for(i=0; i<n; i++)
{
  printf("%d", a[i]);
}
return 0;
}
```

Input	Expected	Got
5 67 34 12 98 78	12 34 67 78 98	12 34 67 78 98

Result:

Thus the given array is sorted using quick sort.

Exp 11

Aim:

To write a program to find the minimum number of Indian currency.

Program:

```
#include<stdio.h>
int main()
{
 int v;
 scanf("%d",&v);
 int d[] = {1000, 500, 100, 50, 20, 10, 5, 2, 1};
 int count =0;
 for(int i=0; i<9; i++)
 {
    count += v/d[i];
    v %= d[i];
 }
 printf("%d", count);
 return;
}
```

Input	Expected	Got
49	5	5

Result:

The program is written and executed.

27/08/26
Exp 12

2-G Cookies problem

Aim:

To maximise the number of content children by assigning cookies of sufficient size.

Program:

```
#include <stdio.h>
int main()
{
    int n, m;
    scanf("%d",&n);
    int g[n];
    for(int i=0; i<n; i++)
    {
        scanf("%d",&g[i]);
    }
    scanf("%d",&m);
    int s[m], u[m];
    for(int i=0; i<m; i++)
        scanf("%d",&s[i]);
        u[i]=0
    int count =0;
    for(int i=0; i<n; i++)
    {
        for(int j=0; j<m; j++)
        {
            if(u[j] ==0 && s[j] >= g[i])
            {
                count ++
                u[j] = 1;
                break;
            }
        }
    }
    printf("%d", count);
}
return 0;
```

Input	Expected	Got
2 1 2 3 1 2 3	2	2

Result:

The program is written and executed successfully.

28/08/26
exp 13

3-Gr Burger Problem

Aim:

To find the minimum distance of a person needs to run by sorting the burgers in descending order.

Program:

```
#include<stdio.h>
#include<math.h>
int main()
{
  int n;
  scanf("%d", &n);
  int arr[n];
  for(int i=0; i<n; i++)
  {
    scanf("%d", &arr[i]);
  }
  int temp;
  for(int i=0; i<n; i++)
  {
    for(int j=i+1; j<n; j++)
    {
      if(arr[i] < arr[j])
      {
        temp = arr[i];
        arr[i] = arr[j];
        arr[j] = temp;
      }
    }
  }
  int sum=0;
  for(int i=0; i<n; i++)
  {
    sum += (pow(n,i) * arr[i]);
  }
  printf("%d", sum);
  return 0;
}
```

27/08/26
exp 14

4- b Array Sum Max Problem

Aim:

To write the program to maximize the sum of given array

Program:

```
#include <stdio.h>
#include <stdlib.h>
int cmp(const void *a, const void *b)
{ return *(int *)a - *(int *)b;
}
int main()
{
  int n;
  scanf("%d", &n);
  int a[n];
  for(int i=0; i<n; i++)
  {
    scanf("%d", &a[i]);
  }
  qsort(a, n, sizeof(int), cmp);
  int sum=0;
  for(int i=0; i<n; i++)
  {
    sum += a[i] * i;
  }
  printf("%d", sum);
  return 0;
}
```

input	Expected	Got
5 2 5 3 4 0	40	40

Result:

The program has written and executed successfully.

24/8/26
exp 15

5-G Product of array elements - Minimum

Aim:

To arrange two arrays such that sum of pairwise products is minimized.

Program:

```
#include <stdio.h>
#include <stdlib.h>
int compare1(const void *a, const void *b){
    return (*(int*)a - *(int*)b);
}
int main()
{
    int n, sum=0;
    scanf("%d", &n);
    int a[n], b[n];
    for(int i=0; i<n; i++)
    for(int j=0; j<n; j++)
    qsort(a, n, sizeof(int), compare1)
    qsort(b, n, sizeof(int), compare2)
    for(int i=0; i<n; i++){
        sum+=(a[i]*b[i]);
    }
    printf("%d", sum);
    return 0;
}
```

input	expected	got
3 1 2 3 4 5 6	28	28

Result:

Thus the program is executed successfully

Exp 16
12/10/26

1 DP

Playing with numbers:

Aim:

To write an efficient algorithm to solve the problem given.

Program:

```
#include <stdio.h>
int main()
{
   int n;
   scanf("%d", &n);
   long long dp[n+1];
   dp[0] = 1;
   for (int i=1; i<=n; i++)
   {
      dp[i] = dp[i-1];
      if (i>=3)
      {
         dp[i] += dp[i-3];
      }
   }
   printf("%lld", dp[n]);
   return 0;
}
```

Sample input	expected	got
6	6	6
25	8641	8641

Result:

Thus the program is executed successfully.

Exp #7
10/10/25

2P

Playing with chessboard

attempt - review.

Aim:

To write the efficient algorithm and execute it for the given problem.

Program:

```
#include <stdio.h>
int main()
{
    int n;
    scanf("%d", &n);
    int a[n][n];
    int dp[n][n];
    for(int i=0; i<n; i++)
    {   for(int j=0; j<n; j++)
        {
            scanf("%d", &a[i][j]);
    } }
    dp[0][0] = a[0][0];
    for(int j=1; j<n; j++)
    { dp[0][j] = dp[0][j-1] +
                 a[0][j];
    }
    for(int i=1; i<n; i++)
    {   dp[i][0] = dp[i-1][0] + a[i][0];
    }
    for(int i=1; i<n; i++)
    {
        for(int j=1; j<n; j++)
        {   if(dp[i-1][j] > dp[i][j-1])
            {   dp[i][j] = dp[i-1][j] +
                           a[i][j];
            }
            else{
                dp[i][j-1] + a[i][j];
            }
    } }
    Printf("%d", dp[n-1][n-1]);
    return 0;
}
```

Input	Expected	Got
3 1 2 4 2 3 4 8 7 1	19	19

Result:

Thus executed successfully.

Exp: 18
10/10/26

3 DP

Longest Common Subsequence

Aim:

To write and execute an efficient algorithm for the given problem.

Program:

```
#include <stdio.h>
#include <string.h>

int main()
{
  char s1[100], s2[100];

  scanf("%s", s1);
  scanf("%s", s2);

  int n = strlen(s1);
  int m = strlen(s2);

  int dp[n+1][m+1];

  for (int i=0; i<=n; i++)
    {
      dp[i][0] = 0
    }

  for (int j=1; j<=m; j++)
  {
    dp[0][j] = 0
  }

  for (int i = 1; i<=n; i++)
  {
    for (int j = 1; j<=m; j++)
    {
      if (s1[i-1] == s2[j-1])
      {
        dp[i][j] = dp[i-1][j-1] + 1;
      }
      else
      {
        dp[i][j] = (dp[i-1][j] > dp[i][j-1])
                   ? dp[i-1][j]
                   : dp[i][j-1];
      }
    }
  }

  printf("%d", dp[n][m]);

  return 0;
}
```

Input	expected	got
aab a2b	2	2

Result:

Thus the given program executed successfully.

Exp: 19
10/10/26

4. DP

Longest non decreasing subsequence.

Aim:

To write an efficient algorithm and execute it for the given algorithm.

Program:

```
#include<stdio.h>
int main()
{
  int n;
  scanf("%d", &n);
  int a[n], dp[n];
  for(int i=0; i<n; i++)
  {
    scanf("%d", &a[i]);
    dp[i] = 1;
  }
  int ans = 1;
  for(int i = 1; i<n; i++)
  { for(int j=0; j<i; j++)
    {
      if(a[j] <= a[i])
      {
        if(dp[j] +1 > dp[i])
        {
          dp[i] = dp[j] + 1;
        }
      }
    }
    if(dp[i] > ans)
    {
      ans = dp[i];
    }
  }
  printf("%d", ans);
  return 0;
}
```

Input	Expected	Got
9 -1 3 4 5 22223	6	6

Results:

Thus the program is executed successfully.

p 20
10/10/26

CP

Duplicates
O(n) : Time complexity O(1)

Space complexity

Aim:

To write and execute an efficient program for the given problem.

Program:

```
#include <stdio.h>
int main()
{
  int n;
  scanf("%d", &n);
  int arr[n];
  for(int i=0; i<n; i++)
  {
    scanf("%d", &a[i]);
  }
  for(int i=0; i<n; i++)
  {
    for(int j=i+1; j<n; j++)
    {
      if(arr[i] == arr[j])
      {
        printf("%d", arr[i]);
      }
    }
  }
  return 0;
}
```

Input	Expected	got
11 10 9 7 6 5 1 2 3 8 4 7	7	7

Result:

Thus the code executed successfully.

Exp 21
19/10/26

2 CP

Finding duplicates

O(n) Time complexity O(1)

Aim:

To write an efficient algorithm and execute it for the given problem.

Program:

```
#include <stdio.h>
int main()
{
    int n;
    scanf("%d", &n);
    int arr[n];
    for(int i=0; i<n; i++)
    {
        scanf("%d", &arr[i]);
    }
    int visarr[100] = {0};
    for(int i=0; i<n; i++)
    {
        if(visarr[arr[i]] == 0)
            visarr[arr[i]] = 1;
        else
            printf("%d", arr[i]);
    }
    return 0;
}
```

Input	Expected	Got
11 10 9 7 6 5 1 2 3 8 4 7	7	7

Result:

Thus the code executed successfully

Exp 22
10/10/26

3 CP

Print intersection of 2 sorted arrays

Aim:

To write an efficient algorithm and execute it for the given problem statement.

Program:

```
#include <stdio.h>
int main()
{
    int T;
    scanf("%d", &T);
    while (T--) {
        int n1, n2;
        scanf("%d", &n1);
        int a[n1];
        for(int i=0; i<n1; i++)
            scanf("%d", &a[i]);
        scanf("%d", &n2);
        int b[n2];
        for(int i=0; i<n2; i++)
            scanf("%d", &b[i]);
        for(int i=0; i<n1; i++)
            for(int j=0; j<n2; j++)
                if(a[i]==b[j])
                    printf("%d", a[i]);
                    break;
        printf("\n");
    }
    return 0;
}
```

Input	Expected	Got
1 3 10 17 57 6 2 7 10 15 57 246	10 57	10 57

Result:

Thus the code executed successfully.

EXP 23
10/10/26

4CP

Pair with difference

Aim:

To write and execute an efficient code for the given problem.

Program:

```
#include<stdio.h>
int main()
{
  int n,k;
  scanf("%d",&n);
  int A[n];
  for(int i=0; i<n; i++)
      scanf("%d",&A[i]);
  scanf("%d",&k);
  for(int i=0; i<n; i++)
      for(int j=i+1; j<n; j++)
          if(A[j]-A[i]==k)
          {
            printf("1");
          } return 0;
  printf("0");
} return 0;
```

Input	Expected	Got
3 1 3 5 4	1	1

Result:

Thus the code executed successfully.

difference

Aim:

To write and execute an efficient algorithm for the given Problem.

Program:

```
#include <stdio.h>
int main()
{
    int n,k;
    scanf("%d",&n);
    int A[n];
    for(int i=0;i<n;i++)
        scanf("%d",&A[i]);
    scanf("%d",&k);
    int i=0,j=1;
    while(i<n && j<n)
    {
        if(i==j)
        {
            j++;
        }
        else if(A[j]-A[i]==k)
        {
            printf("1");
            return 0;
        }
        else if(A[j]-A[i]<k)
        {
            j++;
        }
        else
        {
            i++;
        }
    }
    printf("0");
    return 0;
}
```

Input	Expected	Got
3 1 3 5 4	1	1

Result:

Thus the code ran successfully.

10/1/26

Print intersection of sorted arrays

Aim:

To write and execute an efficient algorithm for the given problem.

Program:

```
#include<stdio.h>
int main()
{
  int T;
  scanf("%d", &T);
  while(T--)
  {
    int m,n;
    scanf("%d",&m);
    int a[m];
    for(int i=0; i<m; i++)
        scanf("%d",&a[i]);
    scanf("%d",&n);
    int b[n];
    for(int i=0; i<n; i++)
        scanf("%d",&b[i]
```

```
int i=0, j=0;
while(i<m && j<n)
{
  if(a[i]==b[j])
  {
    printf("%d", a[i]);
    i++;
    j++;
  }
  else if(a[i]<b[j])
    { i++;
    }
  else{
    j++
    }
  }
 printf("\n");
 return 0;
}
```

Input	expected	got
1 3 10 17 57 6 2 7 10 15 57 246	10 57	10 57

Result:

Thus the code executed successfully.